Spring Boot

Interview Questions and Answers

A Complete Guide for Developers

Topics Covered	Difficulty Level	Total Questions
Core, Web, Data, Security, Cloud	Beginner to Advanced	30+ Questions

Section 1 — Core Spring Boot Concepts

Q1. What exactly is Spring Boot and why do developers love it so much?

Spring Boot is essentially Spring Framework with all the boring setup work done for you. Before Spring Boot existed, you had to write a lot of XML configuration just to get a basic application running. Spring Boot changed all of that by introducing auto-configuration, which means it looks at what libraries you have on your classpath and automatically sets things up for you. It also comes with embedded servers like Tomcat or Jetty built right in, so you can run your application as a standalone JAR file without deploying it to an external server. Developers love it because you can go from zero to a running REST API in just a few minutes.

Pro Tip: Think of Spring Boot as Spring with sensible defaults. You can always override those defaults, but most of the time you do not need to.

Q2. What is the role of the @SpringBootApplication annotation?

The @SpringBootApplication annotation is doing the work of three annotations at once. It combines @Configuration, which tells Spring that this class contains bean definitions. It also includes @EnableAutoConfiguration, which tells Spring Boot to start adding beans based on your classpath settings. And finally it includes @ComponentScan, which tells Spring to look for other components, configurations, and services in the current package and its sub-packages. So when you put @SpringBootApplication on your main class, you are essentially saying go ahead and set everything up automatically.

Q3. How does auto-configuration actually work under the hood?

Auto-configuration in Spring Boot works through a mechanism called conditional beans. When you add a library to your project, Spring Boot checks certain conditions before deciding whether to configure it. For example, if Spring Boot sees that you have added the spring-boot-starter-data-jpa dependency, it will check if you have a DataSource configured. If you do not have one, it will try to create an in-memory H2 database for you automatically. All of this is driven by classes annotated with @Configuration and conditions like @ConditionalOnClass, @ConditionalOnMissingBean, and @ConditionalOnProperty. The auto-configuration classes are listed in a special file called spring.factories or in newer versions, AutoConfiguration.imports.

Pro Tip: You can always see what auto-configuration is being applied by starting your app with the debug flag. Spring Boot will print a detailed report showing what was configured and why.

Q4. What are Spring Boot Starters and why are they useful?

Spring Boot Starters are basically convenience dependency descriptors that you add to your Maven or Gradle build file. Instead of hunting down the exact versions of five different libraries that work well together, you just add one starter and it brings in everything you need. For example, spring-boot-starter-web brings in Spring MVC, an embedded Tomcat server, Jackson for JSON processing, and several other things you need to build a web application. spring-boot-starter-data-jpa brings in Hibernate, Spring Data JPA, and the transaction management you need to work with databases. The beauty of starters is that all the dependencies inside them are tested to work together, so you rarely run into version conflict issues.

Section 2 — Web Development and REST APIs

Q5. What is the difference between @Controller and @RestController?

The @Controller annotation marks a class as a Spring MVC controller, which traditionally means it handles web requests and returns view names that get resolved to HTML templates. When you use @Controller and want to return JSON or plain text directly instead of a view, you need to add @ResponseBody to each method individually. The @RestController annotation was introduced to make this easier. It is simply a shortcut that combines @Controller and @ResponseBody together. So every method in a @RestController automatically writes its return value directly to the HTTP response body as JSON or XML, without trying to resolve it as a view name. For any REST API you build today, you will almost always use @RestController.

Pro Tip: Use @Controller when you are building a traditional web app with Thymeleaf or Freemarker templates. Use @RestController when you are building a REST API that returns JSON.

Q6. How do you handle exceptions globally in a Spring Boot REST API?

The cleanest way to handle exceptions globally is using a class annotated with @ControllerAdvice or @RestControllerAdvice. Inside that class, you define methods annotated with @ExceptionHandler that specify which exception type they handle. When that exception is thrown anywhere in your application, Spring automatically routes it to the right handler method. You can return a custom error response with the appropriate HTTP status code. For example, you might have one method that handles ResourceNotFoundException and returns a 404 response, and another that handles ValidationException and returns a 400 response with details about what went wrong. This keeps all your error handling logic in one place rather than scattered across your controllers.

Q7. What is the purpose of @PathVariable and @RequestParam?

Both annotations are used to extract data from incoming HTTP requests, but they work differently. @PathVariable is used to extract values from the URI path itself. If your URL is /users/42, you would use @PathVariable to capture that 42 as the user ID. @RequestParam is used to extract query parameters, which are the key-value pairs that come after the question mark in a URL. If your URL is /users?page=1&size;=10, you would use @RequestParam to capture the page and size values. A practical way to remember the difference is that path variables are part of the URL structure itself, while request params are optional filters or configuration values added to the URL.

Q8. How does Spring Boot support content negotiation?

Content negotiation is the process of figuring out what format the client wants the response in, whether that is JSON, XML, or something else. Spring Boot supports this primarily through the Accept header that the client sends with its request. If the client sends Accept: application/json,

Spring will serialize the response as JSON. If it sends Accept: application/xml and you have the right library on your classpath, it will return XML. You can also configure content negotiation to work based on file extensions in the URL or request parameters. In most modern REST APIs, JSON is the default, and Jackson handles the serialization automatically because it is included in the spring-boot-starter-web dependency.

Section 3 — Data Access and Persistence

Q9. What is Spring Data JPA and how does it simplify database work?

Spring Data JPA is a layer on top of JPA and Hibernate that dramatically reduces the amount of boilerplate code you need to write for database operations. The most powerful feature is repository interfaces. You create an interface that extends JpaRepository, and Spring Data automatically provides you with implementations for common operations like saving an entity, finding by ID, finding all records, deleting, and counting. Beyond that, you can define methods using a naming convention and Spring Data will automatically generate the right SQL query for you. For example, a method named findByEmailAndStatus will automatically generate a query that filters by both the email and status fields. This means you can build a fully functional data access layer with almost no actual SQL or query writing.

Pro Tip: Spring Data also supports pagination and sorting out of the box. Just pass a Pageable object to your repository method and it handles the LIMIT and OFFSET for you.

Q10. What is the difference between @Entity and @Table annotations?

The @Entity annotation is the fundamental JPA annotation that marks a Java class as a persistent entity, meaning it maps to a database table. Every class you want to store in the database needs this annotation. The @Table annotation is optional and gives you more control over the mapping. You use it when you want to specify the exact name of the database table, or when the table name differs from the class name. For example, if your class is called UserProfile but the database table is called user_profiles, you would put @Table(name = "user_profiles") on the class. If you do not use @Table, JPA will use the class name as the table name by default.

Q11. How do you manage database transactions in Spring Boot?

Spring Boot makes transaction management very straightforward through the @Transactional annotation. When you put this annotation on a method or a class, Spring wraps the execution of that method in a database transaction. If the method completes successfully, the transaction is committed. If a RuntimeException is thrown, the transaction is automatically rolled back. You can customize this behavior with properties on the annotation. For example, you can specify which exception types should trigger a rollback, set the isolation level to control how concurrent transactions interact, and set the propagation behavior to control what happens when one transactional method calls another. In practice, you typically put @Transactional on your service layer methods rather than your repository or controller methods.

Q12. What is the N plus 1 query problem and how do you solve it in Spring Boot?

The N plus 1 problem is one of the most common performance issues in applications that use JPA and Hibernate. It happens when you load a list of entities and then access a related collection on each one, causing Hibernate to fire a separate database query for every single entity. For example, if you load 100 orders and then access the list of items on each order, you end up with 1 query to get the orders plus 100 more queries to get the items, totaling 101 queries. The most effective solutions are using JOIN FETCH in your JPQL queries to load everything in one go, or using the @EntityGraph annotation to specify which relationships to eagerly load for a specific query. You can also use batch fetching by setting the @BatchSize annotation to load related entities in batches rather than one at a time.

Pro Tip: Always enable Hibernate query logging during development so you can see how many SQL queries are being fired. You might be surprised at what you find.

Section 4 — Security and Configuration

Q13. How do you secure a Spring Boot application with Spring Security?

Adding spring-boot-starter-security to your project is all it takes to get basic security in place. The moment you add that dependency, Spring Boot auto-configures HTTP Basic authentication and generates a random password that gets printed to the console. From there, you customize the security by creating a class annotated with @Configuration that extends or uses SecurityFilterChain. In this class you configure which URLs require authentication, which require specific roles, and which are publicly accessible. You also configure how users are authenticated, whether that is from a database, an in-memory list, or an external identity provider. For REST APIs, most teams today use JWT tokens for stateless authentication, which means the server does not need to maintain session state.

Q14. What is the difference between authentication and authorization?

Authentication is the process of verifying who you are. When you log in with a username and password, the system is authenticating you by confirming that the credentials match what it has stored. Authorization is what happens after authentication. It is the process of determining what you are allowed to do. Just because you are authenticated does not mean you can access everything. For example, a regular user might be authenticated but not authorized to access the admin dashboard. In Spring Security, authentication is handled by the AuthenticationManager and its providers. Authorization is handled by the access control rules you define in your security configuration, which might check roles like ROLE_ADMIN or ROLE_USER, or more fine-grained permissions.

> *Pro Tip: A helpful way to remember this is: authentication answers the question of who are you, and authorization answers the question of what are you allowed to do.*

Q15. How do you externalize configuration in Spring Boot and why does it matter?

Externalizing configuration means keeping environment-specific values like database URLs, API keys, and server ports outside of your code so you can change them without recompiling and redeploying. Spring Boot supports this through application.properties or application.yml files, environment variables, command-line arguments, and cloud config servers. The order of precedence matters here. Command-line arguments override environment variables, which override properties files. For different environments like dev, staging, and production, you create profile-specific files like application-dev.properties and application-prod.properties, and then activate the right profile when you start the application. You should never hardcode sensitive values like passwords or API keys in your properties files that get committed to source control. Use environment variables or a secrets manager for those.

Q16. What are Spring Profiles and how do you use them?

Spring Profiles give you a way to have different configurations for different environments without changing your code. You define beans or configuration properties that only activate when a specific profile is active. For example, you might have a DataSource bean that connects to an in-memory H2 database when the dev profile is active, and a completely different DataSource bean that connects to your production PostgreSQL database when the prod profile is active. You activate a profile by setting the spring.profiles.active property, either in your properties file, as an environment variable, or as a command-line argument when starting the application. You can also use @Profile on @Bean methods or entire @Configuration classes to control which components get created in which environment.

Section 5 — Advanced Topics and Best Practices

Q17. What is Spring Boot Actuator and what can you do with it?

Spring Boot Actuator is a sub-project that adds production-ready monitoring and management features to your application with almost no effort. When you add the spring-boot-starter-actuator dependency, you get a set of HTTP endpoints that expose information about your running application. The health endpoint tells you whether your application is up and checks the status of things like database connections and disk space. The metrics endpoint exposes detailed performance data including memory usage, CPU usage, and request counts. The info endpoint lets you expose custom application metadata. The env endpoint shows all your configuration properties. In production, you typically expose only the health endpoint publicly and protect the others or expose them only on an internal management port.

Pro Tip: Actuator integrates seamlessly with monitoring tools like Prometheus and Grafana. This combination gives you powerful real-time dashboards for your application.

Q18. How does caching work in Spring Boot?

Spring Boot provides a simple yet powerful caching abstraction that lets you add caching to your application without tying yourself to a specific caching provider. You enable caching by adding @EnableCaching to a configuration class. Then you use @Cacheable on methods whose results you want to cache. The first time that method is called with a given set of arguments, the result gets stored in the cache. The next time it is called with the same arguments, the cached result is returned directly without executing the method body at all. You use @CachePut to update the cache when data changes, and @CacheEvict to remove entries from the cache. By default Spring Boot uses a simple in-memory cache, but you can easily switch to Redis, Ehcache, or Caffeine by adding the right dependency.

Q19. What is the difference between @Component, @Service, @Repository, and @Controller?

All four of these annotations mark a class as a Spring-managed bean, and from a technical standpoint they all do the same thing at the core level. The difference is semantic, meaning they communicate the intent and role of the class. @Component is the generic stereotype and the parent of the other three. @Service is used on classes in the service layer that contain business logic. @Repository is used on classes that interact with the database, and it has the added benefit of translating database-specific exceptions into Spring's unified DataAccessException hierarchy. @Controller is used on classes that handle web requests in the MVC pattern. Using the right annotation makes your code much more readable and also allows Spring to apply specific behaviors to each stereotype in the future.

Q20. How do you write unit tests and integration tests in Spring Boot?

Spring Boot has excellent built-in support for testing. For unit tests, you typically use JUnit 5 along with Mockito to mock out dependencies so you can test one class in isolation without starting the whole application. For integration tests, Spring Boot provides the @SpringBootTest annotation, which starts up the full application context so you can test how components work together. When testing REST controllers specifically, you can use @WebMvcTest which starts only the web layer without the full application context, making tests faster. For the data layer, @DataJpaTest starts only the JPA components and uses an in-memory database by default. MockMvc is extremely useful for testing controllers because it lets you simulate HTTP requests and verify the response status, headers, and body without starting a real server.

> *Pro Tip: Aim for a good mix of unit tests for business logic and integration tests for testing the components working together. Do not rely only on one type.*

Q21. What are some common performance optimization tips for Spring Boot applications?

There are several areas where you can significantly improve performance. First, be mindful of your JPA queries and always watch out for the N plus 1 problem. Use pagination instead of loading entire tables into memory. Second, make good use of caching for data that does not change frequently. Third, configure a connection pool properly. Spring Boot uses HikariCP by default, which is excellent, but you should tune the pool size based on your workload. Fourth, use asynchronous processing with @Async or message queues for operations that do not need to happen in the request thread, like sending emails or processing images. Fifth, enable lazy loading for application startup by setting spring.main.lazy-initialization to true in environments where startup time matters. Finally, always profile your application under realistic load before optimizing, because premature optimization often targets the wrong things.

Quick Reference — Most Important Annotations

Annotation	Layer	Purpose
@SpringBootApplication	Main Class	Enables auto-config, component scan, and configuration
@RestController	Web Layer	Marks class as REST controller, returns JSON by default
@Service	Service Layer	Marks class as service containing business logic
@Repository	Data Layer	Marks class as data access component
@Entity	Data Layer	Maps Java class to a database table
@Transactional	Service Layer	Wraps method in a database transaction

@Cacheable	Any Layer	Caches the return value of the method
@Async	Service Layer	Runs the method in a separate thread
@Profile	Config Layer	Activates bean only for specific environment profiles
@Value	Any Layer	Injects a value from properties file into a field
@ConfigurationProperties	Config Layer	Binds a group of properties to a typed Java object

---